



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

НАПРАВЛЕНИЕ ПОДГОТОВКИ 09.03.03 Прикладная информатика

О Т Ч Е Т

по индивидуальному заданию

Название: Разработка микросервиса экспорта данных

Дисциплина: Проектирование информационных систем

Студент

ИУ6-75Б

(Группа)

(Подпись, дата)

В. И. Мамыкин

(И.О. Фамилия)

Преподаватель

(Подпись, дата)

А.Ф. Прутик

(И.О. Фамилия)

Москва, 2025

Цель работы: Разработать автономный микросервис для системы "Parser", обеспечивающий экспорт результатов парсинга веб-сайтов в различные форматы данных.

Требования к системе:

Система должна обеспечивать получение списка доступных парсинг-сессий с возможностью просмотра детальной информации о каждой сессии. Необходимо реализовать экспорт данных в три формата: CSV для табличных данных, JSON для структурированного обмена и Excel для удобной работы в офисных приложениях. Каждый экспорт должен фиксироваться в истории с указанием формата, размера файла и количества записей. Сервис должен предоставлять REST API для интеграции с другими компонентами системы и веб-интерфейс для конечных пользователей.

Система должна работать независимо от других микросервисов, обеспечивая время отклика API менее 2 секунд и поддерживая экспорт до 10000 записей за один запрос.

1 Описание решения и используемые технологии:

Для реализации выбрана микросервисная архитектура с использованием контейнеризации. В качестве основного фреймворка бэкенда используется FastAPI на Python 3.11, выбранный за высокую производительность благодаря асинхронности, автоматическую генерацию документации в форматах Swagger и OpenAPI, а также строгую типизацию через Pydantic. База данных PostgreSQL версии 16 обеспечивает надежное хранение данных парсинга и истории экспортов. Для взаимодействия с базой данных применяется ORM SQLAlchemy с асинхронным драйвером asynccpg.

Обработка и экспорт данных реализованы с использованием библиотеки Pandas, которая позволяет эффективно работать с табличными данными и экспортировать их в различные форматы. Для генерации Excel файлов используется библиотека openpyxl в связке с XlsxWriter, что обеспечивает создание файлов с автоматическим подбором ширины колонок для удобства пользователей.

Фронтенд реализован на чистом JavaScript с использованием современных возможностей ES6+, HTML5 и CSS3. Приложение работает как SPA, динамически

обновляя содержимое страницы без перезагрузки. Для раздачи статических файлов и проксирования запросов к API используется веб-сервер Nginx. Вся инфраструктура упакована в Docker контейнеры и управляется через Docker Compose.

Код бэкенда структурирован по слоям для обеспечения поддерживаемости и масштабируемости. API Layer содержит роутеры, обрабатывающие HTTP-запросы и валидирующие входные данные через Pydantic схемы. Service Layer инкапсулирует бизнес-логику экспорта, включая обработку данных через Pandas и генерацию файлов в различных форматах. Data Layer реализован через SQLAlchemy ORM и включает модели ParsingSession, ParsedData и ExportHistory, обеспечивающие работу с PostgreSQL.

2 Архитектурные схемы

На рисунке 1 показана диаграмма контекста, которая показывает систему в окружении, выделяя основные внешние системы и пользователей, которые взаимодействуют с ней. На рисунке 2 – диаграмма контейнеров, показывающая взаимодействие микросервисов и сторонних систем. На рисунке 3 отображена диаграмма компонентов реализованного микросервиса экспорта результатов. На 4 рисунке показана диаграмма классов и структура кода в целом.

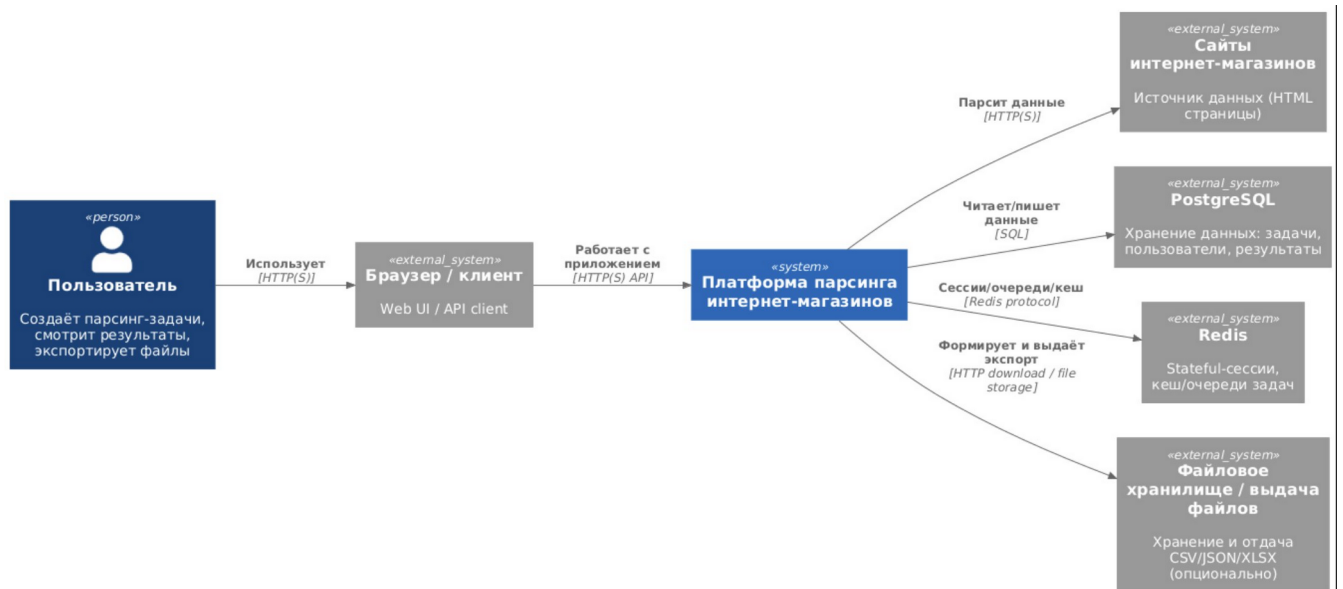


Рисунок 1 – Диаграмма контекста

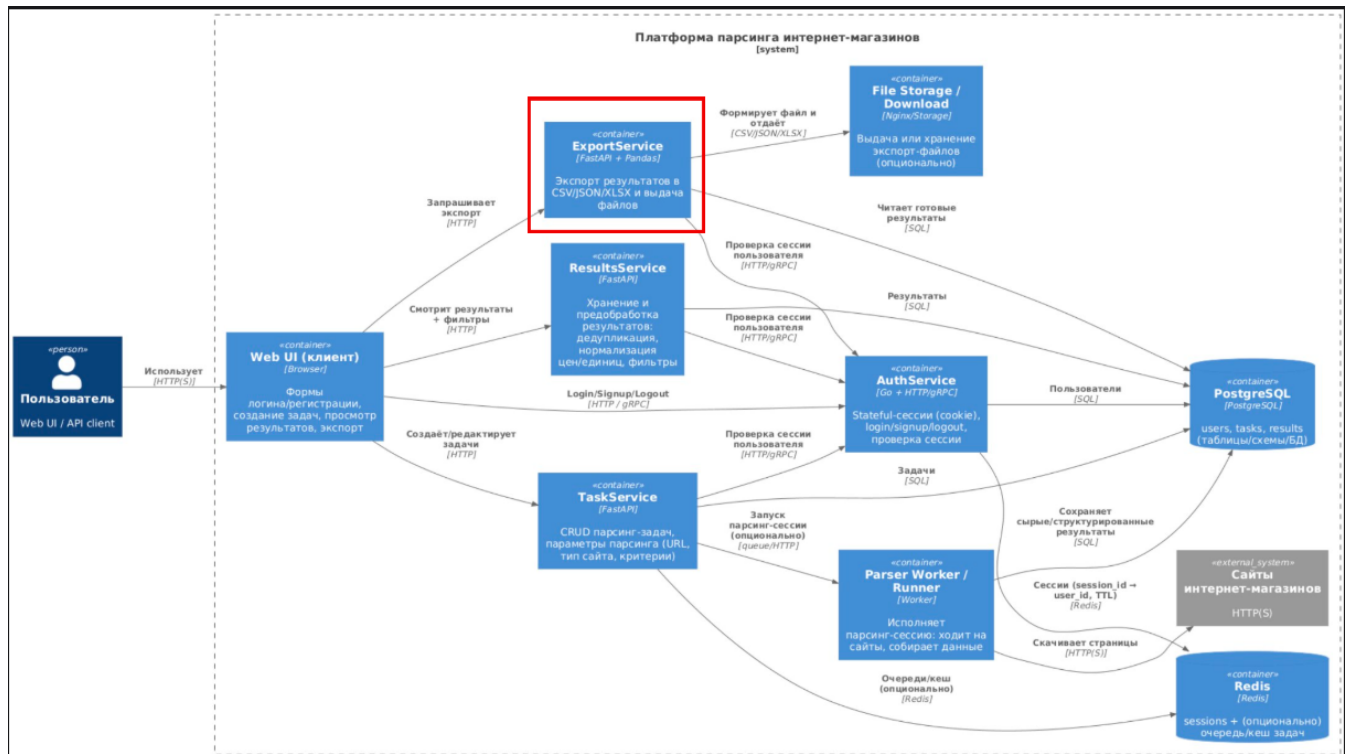


Рисунок 2 – Диаграмма контейнеров

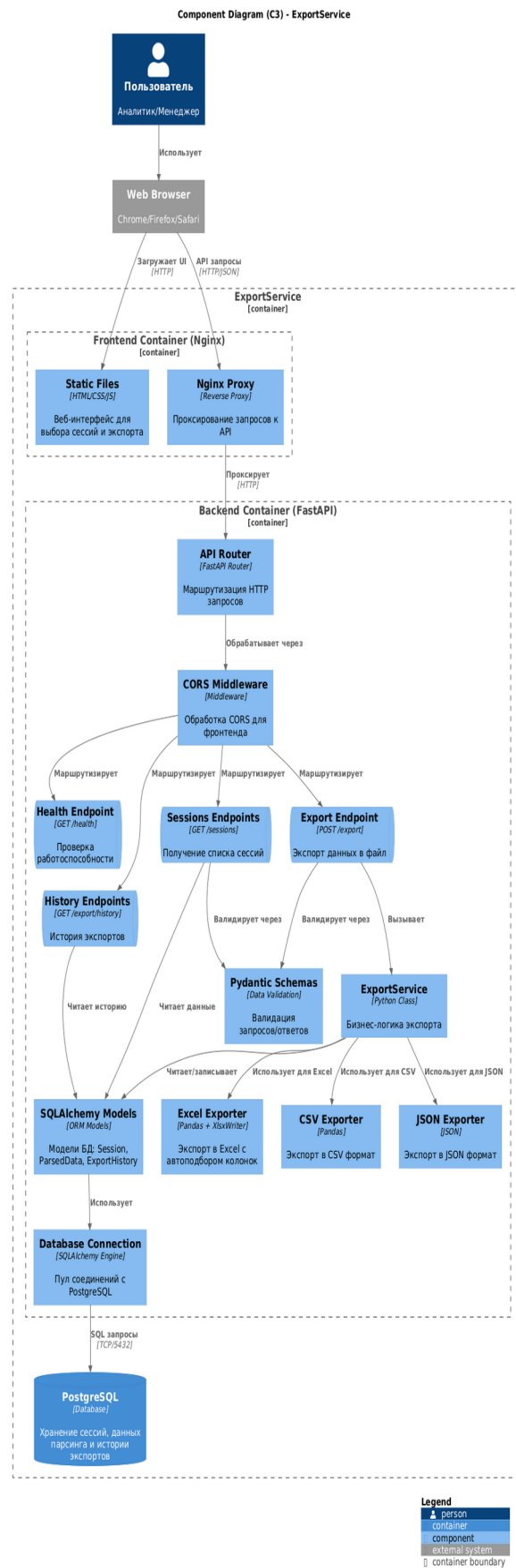


Рисунок 3 – Диаграмма компонентов

Code Diagram (C4) - ExportService Classes

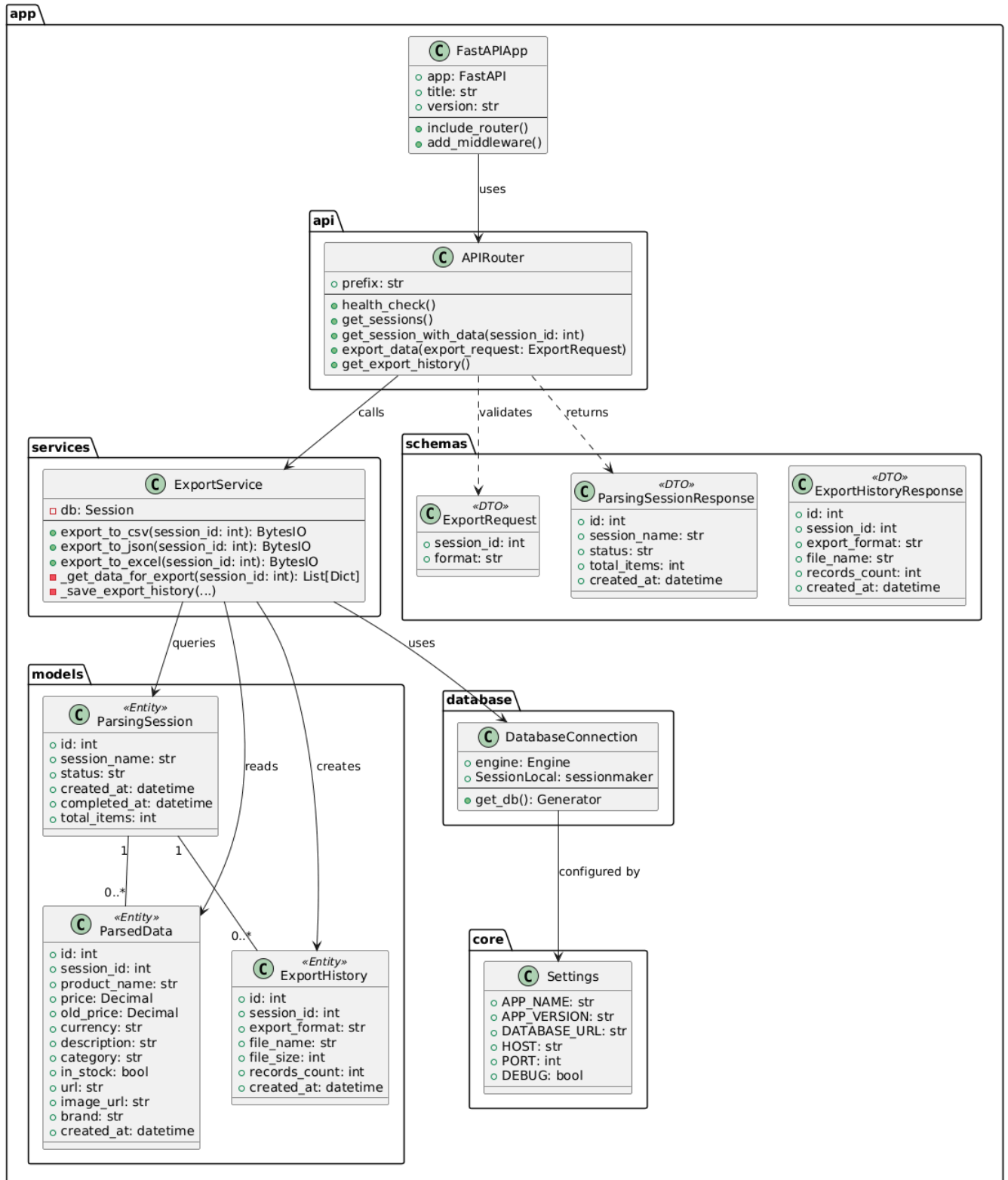


Рисунок 4 – Диаграмма кода

3 Детали реализации:

В ходе работы были реализованы REST API endpoints, обеспечивающие полный функционал сервиса. Endpoint GET /api/v1/health выполняет проверку работоспособности сервиса и подключения к базе данных. GET /api/v1/sessions возвращает список всех доступных парсинг-сессий с информацией о количестве элементов и статусе. GET /api/v1/sessions/{id} предоставляет детальную информацию о конкретной сессии, включая все спарсенные данные для предпросмотра.

Основной функционал экспорта реализован через POST /api/v1/export, который принимает идентификатор сессии и желаемый формат, возвращая файл с данными. Endpoint поддерживает три формата: CSV с кодировкой UTF-8-sig для корректного отображения кириллицы в Excel, JSON с форматированием для удобства чтения и Excel с автоматическим подбором ширины колонок. Каждый экспорт автоматически сохраняется в таблицу export_history с указанием формата, имени файла, размера и количества записей.

Для работы с историей реализованы два endpoint: GET /api/v1/export/history возвращает полную историю всех экспортов, отсортированную по дате, а GET /api/v1/export/history/{session_id} фильтрует историю по конкретной сессии. Это позволяет пользователям отслеживать, когда и в каком формате выполнялись экспорты.

Веб-интерфейс реализован как одностраничное приложение, динамически загружающее данные через Fetch API. При загрузке страницы отображается список доступных сессий с информацией о количестве элементов и дате создания. Пользователь может выбрать сессию для просмотра данных, которые отображаются в виде таблицы с первыми десятью записями. Форма экспорта позволяет выбрать формат и инициировать загрузку файла. История экспортов обновляется автоматически после каждой успешной выгрузки.

Для обеспечения независимости от других микросервисов в базу данных загружаются тестовые данные через init.sql скрипт. Создаются две парсинг-сессии с товарами различных категорий, что позволяет полноценно тестировать

функционал экспорта без необходимости запуска других сервисов системы. Каждый контейнер работает в изолированной Docker сети, что обеспечивает безопасность и предсказуемость поведения.

Особое внимание уделено реализации экспорта в Excel формат, который является наиболее сложным технически. Метод `export_to_excel` использует Pandas для преобразования данных в DataFrame, затем через XlsxWriter создает Excel файл в памяти с помощью BytesIO. Реализован алгоритм автоматического подбора ширины колонок, который анализирует максимальную длину значений в каждой колонке и длину заголовка, выбирая оптимальную ширину с ограничением в 50 символов для предотвращения слишком широких колонок при наличии длинных URL.

Для тестирования создана коллекция Postman, включающая десять тестовых сценариев. Проверяется работоспособность health check endpoint, корректность получения списка сессий и детальных данных, успешность экспорта во всех трех форматах с валидацией Content-Type заголовков, сохранение истории экспортов и корректная обработка ошибочных ситуаций, таких как запрос несуществующей сессии или указание неподдерживаемого формата. Все тесты успешно проходят валидацию, подтверждая корректность реализации.

Развертывание сервиса осуществляется через Docker Compose одной командой, которая поднимает три контейнера: `export_backend` с FastAPI приложением на порту 8000, `export_postgres` с базой данных на порту 5432 и `export_frontend` с Nginx сервером на порту 3000. Автоматически выполняется инициализация базы данных с созданием таблиц и загрузкой тестовых данных. Пользователь получает доступ к веб-интерфейсу по адресу `localhost:3000`, а документация API доступна через Swagger UI по адресу `localhost:8000/docs`.

Вывод: был успешно спроектирован и реализован микросервис экспорта результатов парсинга. Система обеспечивает удобный интерфейс для работы с данными и поддерживает экспорт в три наиболее востребованных формата.